



به نام خدا



دانشگاه تهران
دانشکده مهندسی برق و کامپیوتر
هوش مصنوعی قابل اعتماد

تمرین اول

نام و نام خانوادگی	زکيه امين
شماره دانشجویی	810102398

سؤال اول: تعمیم پذیری

بخش اول : سوالات توضیحی

1. زیربخش اول: مفهوم همبستگی کاذب و شکست ERM

همبستگی کاذب به همبستگی های قوی بین Target label و ویژگی های آسان برای یادگیری اشاره دارد. یعنی مدل به جای تکیه بر ویژگی های واقعا مربوط به Label به این ویژگی های تبعیض آمیز متکی میشه. این الگو های حاشیه ای یا تصادفی در داده آموزش زیاد تکرار میشن اما دلیل واقعی label نیستن. در واقع ERM فقط سعی می کنه میانگین خطا روی کل Training data رو کم کنه.

$$\hat{L}_{avg} = \frac{1}{n} \sum_{(x,y) \in D} l(f_{\theta(x)}, y)$$

مدل های آموزش دیده با ERM اغلب به این ویژگی های تبعیض آمیز تکیه می کنند که باعث پیش بینی های جهت دار و تعمیم پذیری ضعیف روی گروه های اقلیتی میشه که همبستگی کاذب در آنها وجود ندارد. میشه گفت برای ERM مهم نیست الگو واقعی باشه یا نه، صرفا در تلاشه که loss روی داده آموزش کم بشه و این الگوی کاذب به راه میانبره. داده ها به دو گروه bias-aligned و bias-conflicting تقسیم میشن که گروه اول بسیار بزرگتره و در تابع ERM سهم این گروه که همبستگی کاذب درش برقراره از loss خیلی بیشتره. پس گرادیان های ناشی از این ویژگی ها نسبت به ویژگی های درست و واقعی پرتکرارتر می شن و جهت بروزرسانی پارامتر های مدل رو به سمت یادگیری ویژگی های کاذب میبرن. همچنین ERM به Worst group بی توجهه و فقط به میانگین نگاه میکنه. همه این ها باعث میشه تا روی ویژگی های کاذب Overfit بشه.

2. زیربخش دوم: فرموله سازی خطای بدترین گروه و گروه های داده

برای مقابله با همبستگی کاذب، هدف به صورت $\min_{\theta} \max_{b \in B}$ روی بدترین گروه تعریف شده:

$$\min_{\theta} \max_{b \in B} \widehat{L}_b := \frac{1}{n_b} \sum_{(x,y,b) \in G_b} l(f_{\theta}(x), y)$$

این یعنی مدل باید پارامترهای θ را طوری یاد بگیرد که بیشترین میانگین خطا در بین گروه‌ها کمینه بشه و اینطور نباشه که فقط به میانگین خطای کل داده توجه کنه.

تفاوت نمونه‌های هم‌راستا با بایاس و در تضاد با بایاس:

bias-aligned (b_a)

نمونه‌های هم‌راستا با بایاس

نمونه‌هایی که دارای همبستگی کاذب و اتفاقی هستند.

bias-conflicting (b_c)

نمونه‌های در تضاد با بایاس

نمونه‌هایی که فاقد چنین همبستگی کاذبی هستند.

3. زیربخش سوم: نقش تابع زیان GCE

تابع زیان GCE به صورت:

$$l_{GCE}(p, q) = \frac{1 - p_y^q}{q}$$

در داده‌هایی که دچار همبستگی کاذب هستند، Cross Entropy تمایل داره به سرعت روی نمونه‌های هم‌راستا با بایاس Overfit بشه، چون این نمونه‌ها ساده‌ترین و loss آن‌ها زود کاهش پیدا میکنه و گرادیان حاصل از این نمونه‌ها غالب می‌شن و مدل همان ویژگی‌های کاذب را تقویت می‌کند.

تابع GCE با پارامتر q باعث می‌شه اثر نمونه‌های bias-aligned کمتر بشه و در مقابل نمونه‌های bias-conflicting وزن بیشتری بگیرند و مدل به جای تمرکز بر نمونه‌های ساده و هم‌راستا با بایاس، بر نمونه‌های سخت‌تر و در تضاد با بایاس تمرکز کند. در نتیجه اثر بایاس در آموزش کاهش می‌یابد.

4. زیربخش چهارم: بررسی فرض اول مقاله

فرض اول تعریف میکنه که مدل روی گروه در تضاد با بایاس (b_c) به‌طور متوسط خطای بیشتری نسبت به گروه هم‌راستا با بایاس (b_a) داره یعنی:

$$\widehat{L}_{b_a} < \widehat{L}_{b_c}$$

با همین فرض نشان داده می‌شود که مسئله

$$\min_{\theta} \max_{b \in B} \widehat{L}_b$$

به یک کمینه‌سازی خطای وزن‌دار تبدیل می‌شود. وقتی این فرض برقرار باشد، بیشینه‌ی خطای گروهی همیشه خطای گروه (b_c) و هر نمونه وزنی متناسب با احتمال تعلقش به گروه (b_c) می‌گیرد:

$$\min_{\theta} \sum_{(x,y,b) \in D} r(x,y,b) l(f_{\theta}(x), y)$$

از نظر مفهومی، این فرض می‌گه نمونه‌های bias-conflicting سخت‌تر هستند چون همبستگی کاذب در آن‌ها وجود ندارد. بنابراین مدل روی این نمونه‌ها مجبور می‌شه ویژگی‌های واقعی و پایدار را یاد بگیرد، نه سرخ‌های کاذب و تصادفی رو.

5. زیربخش پنجم: ایده الگوریتم JTT

الگوریتم Just Train Twice (JTT) به این صورت است که مدل ابتدا با روش Train ERM می‌شه و سپس نمونه‌هایی که در این مرحله اشتباه طبقه‌بندی شده‌اند (Error set) جدا میشن. در مرحله بعد مدل دوباره آموزش داده می‌شود اما با این تفاوت که به نمونه‌های اشتباه طبقه‌بندی شده وزن بیشتری داده می‌شود (up sampling)، چون فرض بر اینه که از گروه‌های اقلیتی یا سخت هستند. با این کار، مدل نهایی کمتر به همبستگی‌های کاذب وابسته می‌شود و در برابر آن‌ها مقاوم‌تر عمل می‌کند.

بخش دوم: سوالات پیاده سازی

1. آماده سازی داده و مصورسازی

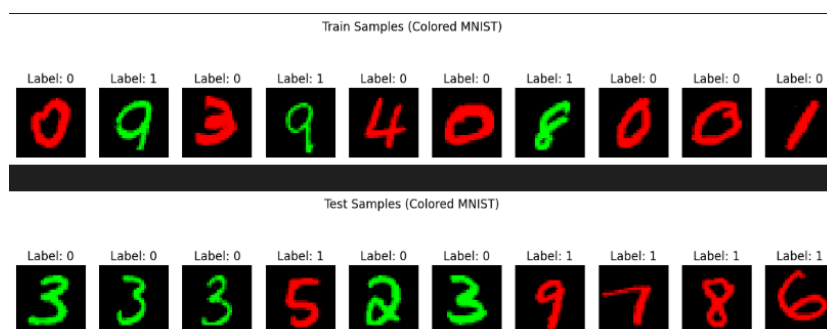


Figure 1_Colored MNIST for train and test dataset

2. آموزش مدل پایه با ERM

Loss در هر epoch به صورت:

Epoch	1	2
Loss	0.0717	0.0657

Table 1_Loss(ERM) per epoch

Loss هر گروه:

Group	0	1
Loss	0.9239	0.9612

Table2_Each group loss

Worst group loss = 0.9612

و دقت مدل روی داده های train و test:

	Train	Test	Difference
Accuracy	0.9797	0.6271	0.3526

Table 3_Accuracy of ERM model

در Train Dataset، همبستگی رنگ و کلاس برابر ۹۵٪ است و مدل با رویکرد ERM یاد می‌گیرد که از Feature رنگ به عنوان میان‌بر استفاده کند. اما در Test Dataset، رنگ برعکس است و چون مدل اشتباه یاد گرفته روی Test عملکرد ضعیف تری نشان می‌دهد. اختلاف Accuracy این دو نشان‌دهنده وابستگی مدل به ویژگی‌های اشتباه (spurious correlations) است.

3. استخراج مجموعه خطا (Error Set)

تعداد نمونه های خطا = 1217

4. ساخت دیتاست وزن دار (Up sampling)

تعداد Sample های موجود در مجموعه داده Up sampled شده = 120,850

$(\text{Train Dataset}) + ((\text{Number of misclassified samples}) * \text{UPSAMPLING_RATE})$

5. آموزش مجدد مدل

Loss در هر epoch به صورت:

Epoch	1	2	3
Loss	0.0070	0.0220	0.0049

Table 4_Loss per epoch

Loss هر گروه:

Group	0	1
Loss	0.6564	0.6598

Table 5_Each group loss

Worst group loss = 0.6598

و دقت مدل روی داده های Train و Test:

	Train	Test	Difference
Accuracy	0.9891	0.7897	0.1994

Table 6_Accuracy of JTT model

گفتیم وقتی یک همبستگی اسپوریوس مثل رنگ در Colored MNIST وجود دارد، مدل از این میانبر استفاده می‌کند. در نتیجه عملکرد روی گروه‌های خلاف این همبستگی ضعیف می‌شود و worst-group loss بالا می‌رود. اما در گام دوم روش JTT نمونه‌هایی که مربوط به گروه‌های سخت یا اقلیت هستند وزن بیشتری می‌گیرند. مدل مجبور می‌شود ویژگی‌های واقعی‌تر مثل شکل رقم را یاد بگیرد. به همین دلیل JTT معمولاً accuracy بهتر و worst-group loss کمتری نسبت به ERM دارد.

6. پیاده سازی تابع زیان GCE

$$q = 0.7$$

تعداد نمونه های خطا = 2344

Loss در هر epoch به صورت:

Epoch	1	2
Loss	0.0451	0.409

Table 7_Loss per epoch

Loss هر گروه:

Group	0	1
Loss	12.5821	9.5367

Table 8_Each group loss

Worst group loss = 12.5821

و دقت مدل روی داده های train و test:

	Train	Test	Difference
Accuracy	0.9609	0.2885	0.6724

Table 9_Accuracy of GCE

تابع هزینه GCE برای مقابله label noise طراحی شده و تلاش می کند تأثیر نمونه هایی را که به نظر سخت هستند کاهش بدهد. اما در Colored MNIST مشکل اصلی همبستگی کاذب است و نه نویز برچسب. پس نمونه هایی که به شکستن این همبستگی کمک می کنند ممکن است توسط GCE کم اهمیت در نظر گرفته بشه و مدل همچنان به میانبرهایی مثل رنگ تکیه کند.

سؤال دوم: حملات خصمانه

بخش اول: مرور مفاهیم

روشن جعبه سفید	Attacker همه چیز رو راجب مدل میدونه اعم از معماری، هایپارامترها، وزن ها و
روشن جعبه سیاه	Attacker فقط به ورودی و خروجی دسترسی داره و داخل مدل رو نمیبینه. پس یه Sample رو هی با Noise های رندوم دستکاری میکنیم. Dataset بدست آمده را روی یه مدل دوم Train میکنیم و مدل دوم عملاً میشه تخمینی از مدل اول حوالی Sample و به رویکرد جعبه سفید میرسیم.

Table 10_White/Black attacks definition

- در اکثر مدل ها Attack به صورت جعبه سیاه هست و این مدل ها نشون میدن که مدل چقدر آسیب پذیره و باید دید چگونه بدون دسترسی به داخل مدل میتون آن را فریب داد. بنابراین میتون ضعف های مدل را تشخیص و امنیت مدل را افزایش داد.

•

Targeted Attack	یه سمپل Adversarial به مدل میدهیم و انتظار داریم خروجی مدل برچسب خواسته ما باشد.
Non Targeted Attack	یه سمپل Adversarial به مدل میدهیم و انتظار داریم خروجی مدل برچسب درست آن نباشد. فقط میخوایم مدل اشتباه کنه.

Table 11_Targeted/Non targeted Attacks definition

بخش دوم: حمله جعبه سیاه

- اولاً که Attacker به مدل و ساختارش دسترسی داره، همچنین تو مقاله اشاره شده که Image ها اصولاً نزدیک Decision boundary قرار دارن و یک تغییر کوچک و نامحسوس هم میتونه Label رو عوض کنه.
- در حالت Black box چون به گرادیان و داخل مدل دسترسی ندارن و فقط میتون از Confidence score خروجی استفاده کنن از بردارهای متعامد استفاده میشه. ایده کلی حمله = نمونه‌های طبیعی به Decision boundary نزدیک‌اند. لازم نیست جهت دقیق گرادیان را بدانیم. با آزمون جهت‌های متعامد تشخیص میدیم به مرز تصمیم نزدیک می‌شویم یا نه و تصویر را گام‌به‌گام به آن سمت حرکت می‌دهیم.
- SimBA در فضای پیکسلی perturbation را با دستکاری مقادیر پیکسل‌ها در جهت‌های تصادفی متعامد اعمال می‌کنه.

روش SimBA-DCT بر مبنای SimBA است، اما perturbation رو در فضای DCT اعمال می‌کنه (DCT یک تبدیله که اطلاعات تصویر را از دامنه‌ی فضایی به دامنه‌ی فرکانسی منتقل می‌کنه). با تمرکز روی فرکانس‌های پایین که اطلاعات ساختاری اصلی تصویر رو دارن تغییرات اعمال شده نامحسوس‌تر و باکیفیت‌تر خواهند شد و حملات adversarial موثرتر و با کیفیت بصری بهتر منجر می‌شود.

- در Score-based attacks مهاجم به confidence score یا خروجی پیوسته‌ی مدل دسترسی داره. در حالی که در Label-based attacks مهاجم فقط به label پیش‌بینی‌شده دسترسی دارد و گرادیان را نمی‌دونه.

Average queries	میانگین تعداد Query ها تا موفقیت حمله
Average L2	میانگین اندازه perturbation در نورم L_2
Success rate	نرخ موفقیت حمله

Table 12_Evaluation metrics

بخش سوم: پیاده سازی حملات جعبه سفید

1. زیربخش اول: مدل و مجموعه داده

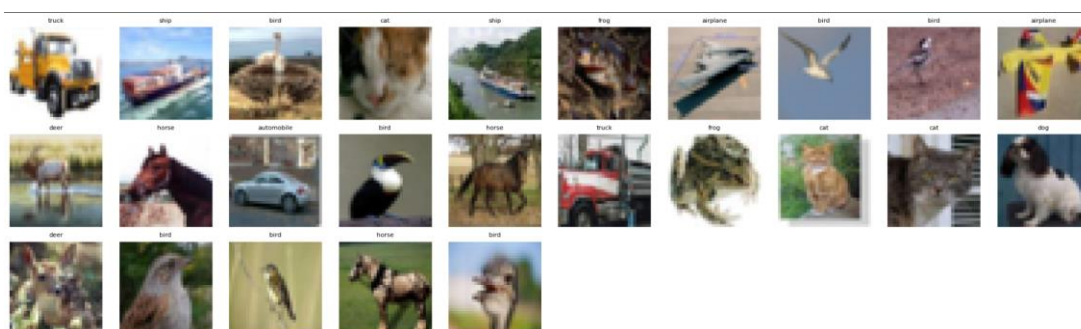


Figure 2_ CIFAR_10 classes

Accuracy of the model on the 25 clean images: 1.00

2. زیربخش دوم: FGSM

روش FGSM یا همون Fast Gradient Sing Method از دسته حملات خصمانه است که با یه تغییر کوچیک توی ورودی در تلاشه تا مدل رو گول بزنه و مدل خروجی اشتباه بدهد.

یعنی برای بیشینه کردن خطای مدل، ورودی رو در جهت گرادیان خطا کمی حرکت بدیم. یه Step میاد رو Loss function. Gradient descent اعمال میکنه. همسایگی رو L_∞ در نظر میگیریم با شعاع ϵ و داریم:

Non-targeted attack: $x_{adv} = x + \epsilon \text{sign}(\nabla_x l(\theta, x, y))$

Targeted attack = $x_{adv} = x - \epsilon \text{sign}(\nabla_x l(\theta, x, \hat{y}))$

Accuracy پس از حمله با روش FGSM به 0.280 کاهش پیدا کرد که نشان می دهد حمله موفق بوده.

با توجه به Result مشخص شده در تصویر پایین میبینیم که تغییرات (Noises) برای چشم انسان نامحسوس است:

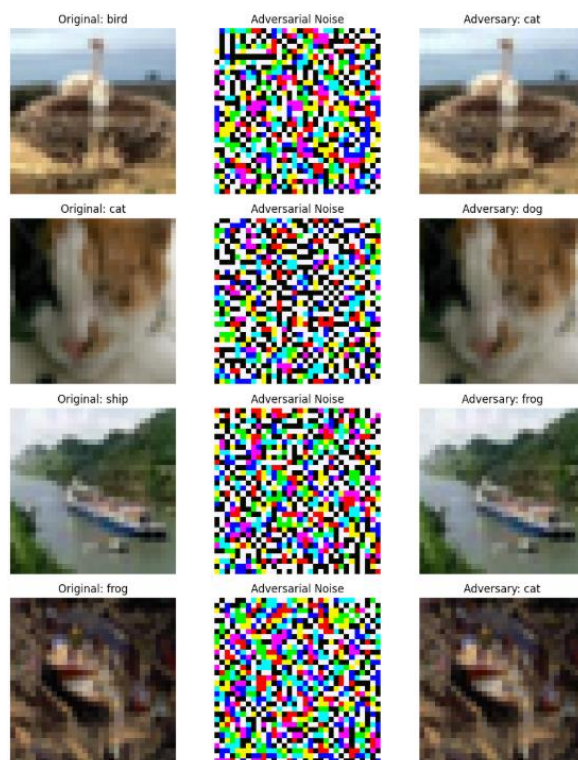


Figure 3_FGSM Adversarial sample

3. زیربخش سوم: PGD

روش PGD یا همون Projected Gradient Descent برعکس FGSM که فقط یه بار از گرادیان استفاده می کنه این کار رو چندین بار و به صورت تکراری انجام می ده و در نظر میگیره اگه پس از Gradient Descent از Feasible region خارج شد دوباره تصویر شه (Project) به داخل Feasible region.

مطابق صورت پروژه داریم: $T = \max(20, \alpha\varepsilon)$

Non-targeted attack: $x'_{t+1} = \text{proj}\{x'_t + \alpha \text{sign}[\nabla_x l(\theta, x'_t, y)]\}$

Targeted attack: $x'_{(t+1)} = \text{proj}\{x'_t - \alpha \text{sign}[\nabla_x l(\theta, (x'_t), y)]\}$

Accuracy پس از حمله با روش PGD به 0.12 کاهش پیدا کرد که نشان می دهد حمله

موفق بوده. با توجه به Result مشخص شده در تصویر پایین میبینیم که تغییرات (Noises)

برای چشم انسان نامحسوس است:

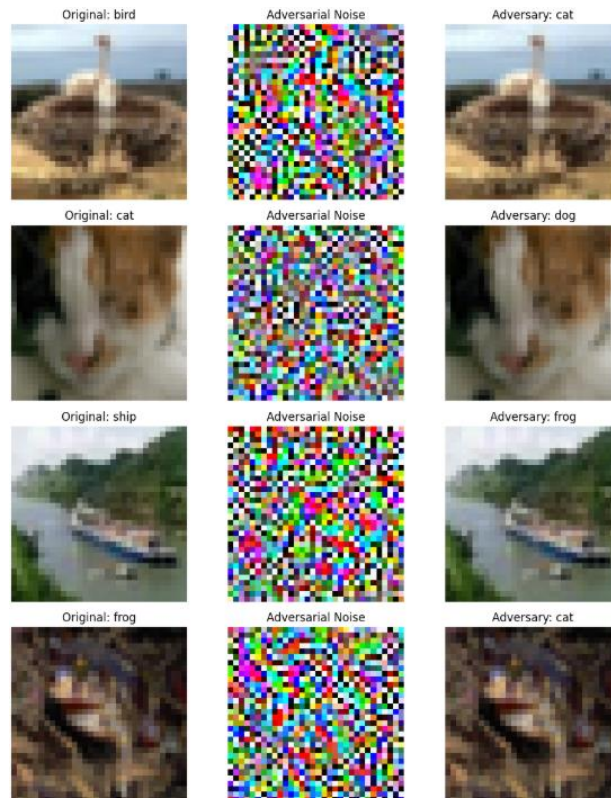


Figure 4_PGD Adversarial sample

